

JOINT DELIVERY ARCHITECTURE

Koard + Decionis

Agentic card authorization

Cardholder-present verification before card use, bound to a purchase intent and enforced at the bank's authorization boundary.

The joint delivery

Koard is the design partner for cardholder-present verification. Decionis provides independent execution authority. AgentSafe enforces the grant boundary in the bank; the issuer retains the final card decision.

PARTICIPANT	OWNS	OUTPUT
Agent platform	Propose the precise purchase; wait before presenting the card.	Purchase intent
Koard	Verify cardholder presence and bind the intended card to the purchase.	Signed presence evidence
Decionis	Verify evidence and apply the bank's versioned policy.	Decision and bound grant
AgentSafe	Capture, hold, match, claim once and record effect evidence.	Intent approval / NO_MATCH
Issuer / processor	Operate the hook and account/fraud controls; authorize or decline.	Card decision and result

Three phases, with one clear point of execution

Before card use: establish intent and required presence, then obtain authority.

At authorization: match and claim within the issuer's measured deadline.

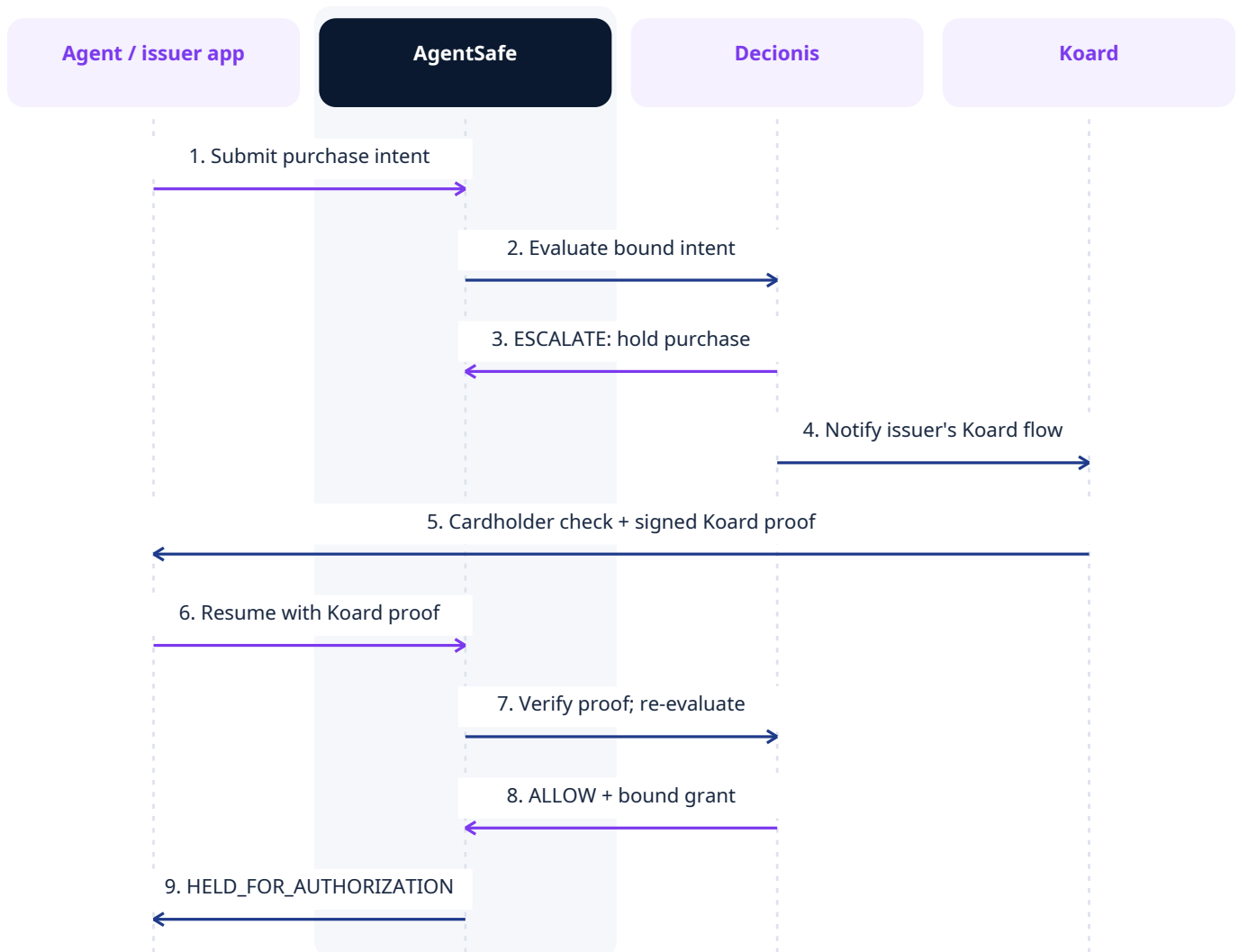
Afterward: record what the issuer actually did and reconcile uncertainty.

A human is awaited only before card submission. The live authorization path never waits for a tap.

PHASE 1 | BEFORE CARD USE

Verify presence, then decide

The original purchase stays immutable. Koard supplies evidence for that purchase; only a fresh policy decision can issue an executable grant.



Koard signs the verification result and returns it through the agreed issuer-app integration. The bank's escalation subscription carries the exact intent identity, expiry and purchase fields needed for the ceremony. A bank-role approval uses Decionis Presence instead.

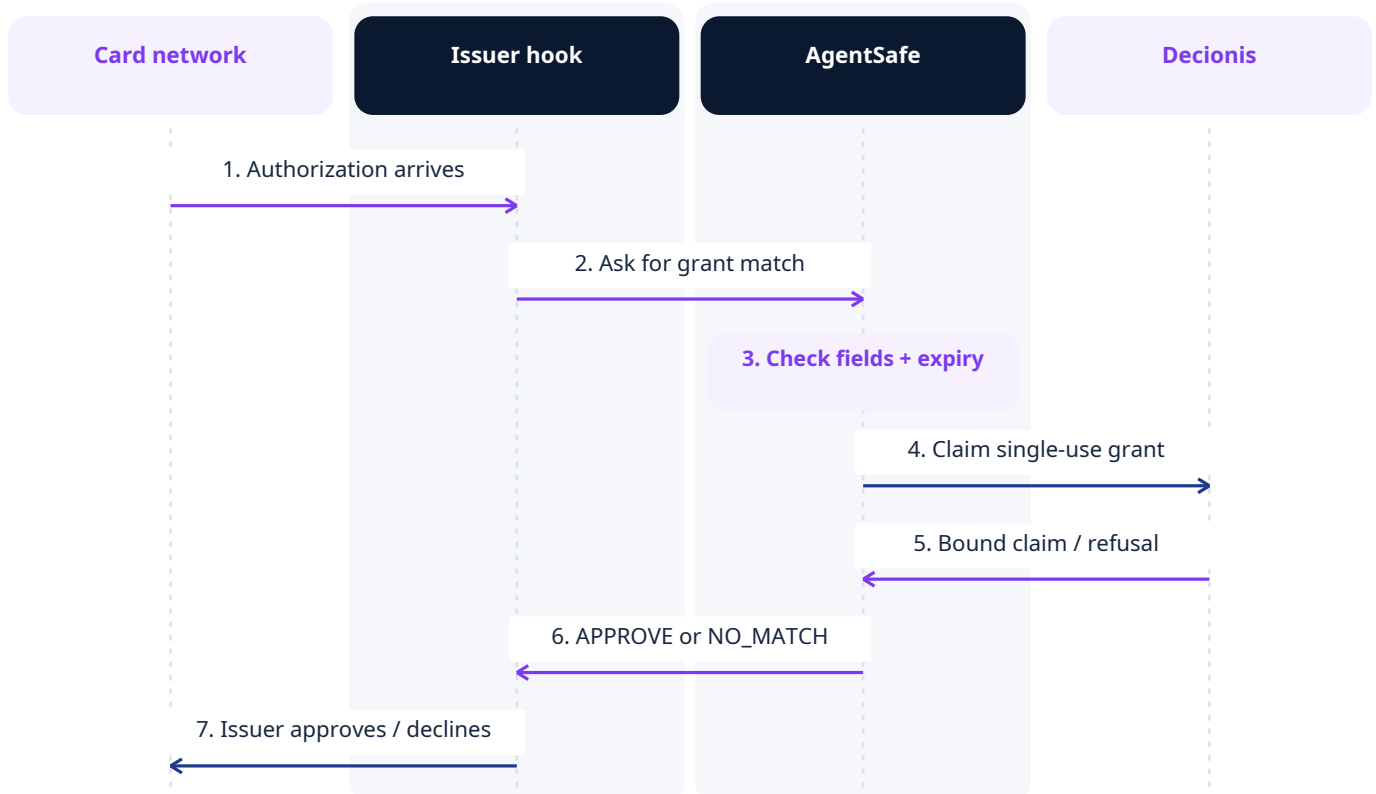
No permission is created by a tap alone

BLOCK stops submission. ESCALATE stays held. If the intent or evidence expires, obtain a fresh intent and approval as required. Successful presence must satisfy the bank's policy before ALLOW.

PHASE 2 | ISSUER AUTHORIZATION

Match locally. Claim once.

The issuer asks about the authorization it received. The matcher runs in the bank; claiming the grant is a call to the configured authority.



Every match must satisfy the same purchase

Exact issuer token reference, currency and merchant id; exact merchant category when the purchase specified one; positive amount no greater than the granted ceiling; unexpired, unused grant. The grant is cryptographically verified and claimed against its original intent.

APPROVE is the intent leg of the issuer's decision

The issuer still applies funds, fraud and other card controls. NO_MATCH means usable authority was not established; it can reflect missing state, mismatch, expiry or claim failure. It does not prove the cardholder was absent.

Agree a deadline for the complete hook, including claim I/O. No latency guarantee is established by a local lookup. Define an explicit NO_MATCH / timeout policy; strict agentic enforcement must not silently bypass a required grant.

CONTRACTS | INTEGRATION HANDOFFS

Name the receiving boundary

These are separate interfaces. The issuer's hook calls AgentSafe's trusted executor; it does not call the generic proxy's control routes.

CALLER / RECEIVER	CONTRACT	PURPOSE
Agent to AgentSafe	POST /v1/actions PROPOSER	card.purchase with target card:<token ref>
AgentSafe to Decionis	POST /v1/authority/ enforce-and-bind	Evaluate the captured intent; return a decision and grant if allowed.
Decionis to subscriber	authority.escalation_required	Notify the bank-configured endpoint for the Koard journey.
Issuer app to AgentSafe	POST /v1/escalations PROPOSER	Resume the same intent with the signed presence attestation.
Issuer hook to AgentSafe	POST /v1/card-authorizations OPERATOR: cards.authorize	Match request fields; claim one bound grant.
AgentSafe to Decionis	POST /v1/execution/claim-token	Consume authority once, subject to validity and current checks.
Issuer hook to AgentSafe	POST /v1/card-authorizations/ {authorization_id}/result	Report the issuer result for comparison and finalization.

Bind the same identifiers through every handoff

Carry tenant, intent id/hash, decision, dossier and grant bindings. Use a stable issuer authorization id. Identical concurrent retries share one claim and answer; a changed request under the same id is refused before a second card is claimed.

The actor submitting a purchase must not also hold issuer-authorization authority. Configure separate principals and restrict the processor credential to cards.authorize. Transport authentication does not replace the intent and grant checks.

EVIDENCE | PRESENCE AND EFFECT

Keep proof and outcome distinct

Koard proves presence for the purchase. The issuer reports the card decision. Decionis records the authority decision; AgentSafe links the observed effect.

Presence attestation: the Decionis-side contract

Compact JWS with alg=EdDSA, typ=decionis-presence-attestation+jwt and the registered kid. The bank admits Koard's key and issuer identity. Align Koard's native output with this contract in the joint integration.

CLAIMS	BINDING
iss, aud, sub	Registered provider; bank organization; opaque cardholder/token reference, never a PAN.
intent_id, intent_hash	The exact purchase being approved. Another purchase requires its own bound evidence.
iat, exp, jti	Issued time, expiry and unique attestation id; declared lifetime at most 300 seconds. Agree clock-skew tolerance.
presence	method=card_tap; result=VERIFIED; approver_role=cardholder. Optional card_token_ref, token_match and network_validated.

Issuer result: the current AgentSafe endpoint

```
POST /v1/card-authorizations/auth_ref_demo/result
{
  "status": "APPROVED",
  "approved_amount_minor": 4000,
  "auth_code": "A7"
}
```

The endpoint authenticates an issuer operator and accepts structured JSON. It does not accept an issuer-signed result JWS. AgentSafe builds effect evidence from this report and submits finalization; a cryptographic issuer receipt requires a separately agreed connector contract.

Approved amounts within the purchase ceiling match; excess is an effect mismatch and halts by default. DECLINED records failure. A result after the claim lease is indeterminate, with reconciliation required.

JOINT DELIVERY | ACCEPTANCE

Close the issuer integration

The core card flow exists in AgentSafe source. Production acceptance depends on the Koard ceremony, bank policy and the processor's actual authorization hook working together.

WORKSTREAM	ACCEPTANCE EVIDENCE
Koard + issuer app	Demonstrate the purchase shown to the cardholder, proof-to-card/token binding, signed handoff, failed/abandoned verification, expiry and key rotation.
Decionis + bank risk	Register the producer key; admit the required presence facts; prove re-evaluation after evidence; test mismatched tenant/intent and revoked keys.
Issuer / processor	Agree token and merchant mapping, real-time hook, deadline, NO_MATCH policy, duplicate authorization handling and final result reporting.
Bank platform	Separate identities, restrict bypass paths, require durable evidence, map replicas to card state, exercise restart and recovery.
Joint validation	Run normal purchase, BLOCK, ESCALATE, changed amount/merchant, expired grant, duplicate claim, identifier collision and unknown outcome end to end.

Current limits that shape the rollout

One spendable hold per card per process. Holds and retry lookup are in memory; an execution journal does not restore that lookup. Partial amounts can match the ceiling once; repeated incremental or split authorizations need an explicit design. Velocity facts must be supplied by the issuer.

Deployment can place the decision service in an agreed hosted or bank-controlled environment; evaluate the actual licensed configuration. The reviewed card flow still claims against its configured authority. Do not assume edge policy evaluation removes that dependency.

Review basis: the supplied 3 October briefs, the design-partner context, AgentSafe master 2e564711 and the accompanying hardening changes. Decionis-side attestation and webhook contracts were checked in local source; live service availability and issuer integration were not tested.